Applications of Binary Search

September 23, 2023

2023 · # rust, # algorithms · notes

Binary search is undoubtedly one of computer science's most well-known and fundamental algorithms. This elegant and efficient algorithm searches for a given key within a sorted array of n items.

Binary search repeatedly divides the search range in half until the target element is found or the search range becomes empty, resulting in a time complexity of $\Theta(\log n)$. This is one of the easiest applications of the *Divide-and-Conquer paradigm*.

Divide-and-Conquer Paradigm

The divide-and-conquer paradigm tackles a complex problem by breaking it down into smaller, more manageable subproblems of the same type. These subproblems are addressed recursively, and their solutions are combined to yield the solution for the original problem.

More precisely, a divide-and-conquer-based algorithm follows three main steps:

- **Divide**: The initial problem instance is partitioned into smaller subinstances of the same problem.
- **Solve**: These subinstances are then solved recursively. If a subinstance reaches a certain manageable size, a straightforward approach is employed to solve it directly.
- **Combine**: The solutions obtained from the subinstances are combined to obtain the final solution for the original, larger instance of the problem.

Binary Search Implementation

We can apply the above paradigm to search for a key in a sorted array of n elements within $\Theta(\log n)$ comparisons.

- **Divide**: The array is divided into two roughly equal halves, centering around the middle element of the array.
- **Solve**: Compare the middle element of the array with the searched key. If the middle element is a match, the search stops successfully. If not, we recursively search for the key only in one of the two halves that may contain based on whether the desired key is greater or lesser than the middle element.
- **Combine**: There is nothing to combine. The algorithm simply reports the final answer.

A Rust implementation of binary search is the following.

```rust
fn binary_search<T: Ord>(arr: &[T], key: T) -> Option<usize> {
    let mut low = 0;
    let mut high = arr.len();

    while low < high {
        let middle = low + (high - low)/2;

        match key.cmp(&arr[middle]) {
            std::cmp::Ordering::Equal   => return Some(middle),
            std::cmp::Ordering::Less    => high = middle,
            std::cmp::Ordering::Greater => low = middle + 1,
        }
    }
    None
}
```

The generic implementation above works for types that are `Ord`. `Ord` is the trait for types that form a total order. The method `cmp` returns an `Ordering` between two elements: In our case, the `key` we are looking for and the element in the middle. We use the result of this comparison to check for a match or to move either `low` after `middle` or `high` to `middle`. Note that the position `high` is not included in the range.

It is worth noticing the expression `middle = low + (high - low)/2` to compute the position in the middle of the current range. A lot of existing implementations on the net use instead the expression `middle = (low + high) / 2`, which is buggy. Indeed, it leads to

It is also important to observe that when there are multiple occurrences of the searched key, the function returns the position of the first encountered occurrence, not necessarily the first occurrence in the vector. This behavior aligns with the implementation of binary search in the Rust Standard Library. However, it is often very useful to report the position of the first (or last) occurrence of the searched key. We can obtain this behavior with the following implementation.

```rust
fn binary_search<T: Ord>(arr: &[T], key: T) -> Option<usize> {
    let mut low = 0;
    let mut high = arr.len(); // note that high is excluded

    let mut ans = None;

    while low < high {
        let middle = low + (high - low) / 2;

        match key.cmp(&arr[middle]) {
            std::cmp::Ordering::Equal => {
                ans = Some(middle);
                high = middle
            }
            std::cmp::Ordering::Less => high = middle,
            std::cmp::Ordering::Greater => low = middle + 1,
        }
    }

    ans
}
```

In this implementation, when a match is found, we do not immediately return its position. Instead, we update the `ans` variable and set `high` to the position of this occurrence. This way, we continue the search in the first half of the array, seeking additional occurrences of the `key`. If there are more matches, `ans` will be further updated with smaller positions.

As a useful exercise you could try to modify the code above to return the smallest position such that the element at that position is greater than or equal to `key`. In other word, if the `key` is not in the slice, it returns the position of its successor.

Instead of implementing the code above, we can find the first (or even the last) occurrence of a key with partition_point method of the standard library. This method is even more generic that our code above. Indeed, it returns the index of the partition point in a sorted vector according to any given predicate.

Binary Search the Answer

Consider a problem where all the possible candidate answers are restricted to a range of values between certain `low` and `high` possible answers. In other words, any candidate answer x falls within the range `[low, high)`. We also have a boolean predicate `pred` defined on the candidate answers that tells us if an answer is good or not for our aims. Our goal is to find the largest good answer.

When no assumptions are made about the predicate, we cannot do better than evaluating the predicate on all the possible answers. So, the number of times we evaluate the predicate is $\Theta(n)$, where $n = high - low$ is the number of possible answers.

Instead, if the predicate is **monotone**, we can *binary search the answer* to find it with $\Theta(\log n)$ evaluations. This strategy is implemented by the generic function below.

```rust
use num::FromPrimitive;
use num::Num;
use std::cmp::PartialOrd;

fn binary_search_range<T, F>(low: T, high: T, pred: F) -> Option<T>
where
    T: Num + PartialOrd + FromPrimitive + Copy,
    F: Fn(T) -> bool,
{
    let mut low = low;
    let mut high = high;

    let mut ans = None;

    while low < high {
        let middle = low + (high - low) / FromPrimitive::from_u64(2).unwrap();

        match pred(middle) {
            true => {
                low = middle + T::one();
                ans = Some(middle)
            }
            false => high = middle,
        }
    }

    ans
}
```

The function takes the extremes (of type `T`) of the range and the predicate as an argument. We use the external crate Num to require some basic arithmetic operations for type `T`. The function returns the largest element of the range satisfying the predicate, or `None` if there is no such element.

Let's use this function to solve problems.

Sqrt

An example is the problem *Sqrt*.

We are given a non-negative integer v and we want to compute the square root of v rounded down to the nearest integer.

The possible answers are in $[0, v]$. For each candidate answer x, the predicate is $p(x) = x^2 <= v$. Thus, we can find the result in $\Theta(\log v)$ time.

Thus, a one-line solution is

```rust
fn sqrt(v: u64) -> u64 {
    binary_search_range(0, v + 1, |x| x * x <= v).unwrap()
}
```

Social Distancing

Let's consider another problem.

*We have a sequence of n mutually-disjoint intervals. The extremes of each interval are non-negative integers. We aim to find c integer points within the intervals such that the smallest distance d between consecutive selected points is **maximized**.*

Guess what? A solution to this problem binary searches the answer, the target distance d. Why is this possible? If a certain distance is feasible (i.e., there exists a selection of points at that distance), then any smaller distance is also feasible. Thus, the feasibility is a monotone boolean predicate that we can use to binary search the answer.

As the candidate answers range from 1 to l, where l is the overall length of the intervals, the solution takes $\Theta(\log l)$ evaluations of the predicate.

What's the cost of evaluating the predicate? Well, we first sort the intervals. Now, we can evaluate any candidate distance d' by scanning the sorted intervals from left to right. First, we select the left extreme of the first interval as the first point. Then, we move over the

predicate takes $\Theta(n)$ time.

The overall running time is $\Theta(n \log l)$.

A Rust implementation of this strategy is the following.

```rust
fn select_intervals(intervals: &mut Vec<(usize, usize)>, c: usize) -> Option<usize> {

    let l = intervals
        .iter()
        .fold(0, |acc, interval| acc + interval.1 - interval.0 + 1); // overall length

    if l < c {
        // there is no solution
        return None;
    }

    intervals.sort_unstable();

    // A closure implements our predicate
    let pred = |d: usize| -> bool {
        let mut last_selected = intervals[0].0;
        let mut cnt = 1;
        for &interval in intervals.iter() {
            while interval.0.max(last_selected + d) <= interval.1 {
                last_selected = interval.0.max(last_selected + d);
                cnt += 1;
            }
        }

        cnt >= c
    };

    binary_search_range(1, l + 1, pred)
}
```

Other Problems

- Find First and Last Position of Element in Sorted Array
- Find the minimum in a rotated sorted array
- Search for a peak in an (unsorted) array

These notes are for the *"Competitive Programming and Contests"* course at Università di Pisa.

Enjoy Reading This Article?

Here are some more articles you might like to read next:

- HandsOn 3
- HandsOn 2
- HandsOn 1
- Mo's Algorithm
- Dynamic Prefix Sums with Fenwick Tree